Azure AI Search vs SQL Server 2025 Hyperscale

Native vector + semantic search comparison · cost crossover model · migration map · East US pricing, May 2026

Can SQL Server 2025 replace Azure AI Search with zero effort?

No. SQL Server 2025 Hyperscale gives you the *primitives* for a retrieval system — the `vector` data type, DiskANN (preview), Full-Text Search, statistical semantic key phrases. Azure AI Search is a *turnkey retrieval platform*. To get from primitives to platform there is meaningful upfront engineering, ongoing operational overhead, and a small set of capabilities that don't have a SQL equivalent at all.

What does NOT port at all (no SQL equivalent — must approximate or live without)

AI SEARCH FEATURE	CLOSEST SQL SUBSTITUTE	PARITY
Microsoft-tuned semantic re-ranker (L2)	Self-host a cross-encoder (e.g. ms-marco-MiniLM) on Container Apps + GPU, or call AOAI gpt-4o-mini for LLM re-rank. Quality and per-query latency will differ.	Approximate only
Agentic retrieval (autonomous query planner)	Orchestrate from app tier with Semantic Kernel or LangGraph against your SQL hybrid search. Built, not bought.	Build yourself
Integrated vectorization (one-click chunk + embed at index time)	External pipeline calling AOAI embeddings, or `sp_invoke_external_rest_endpoint` per row. No UI, no managed orchestration.	Build yourself
Indexer connector library (Blob, ADLS, Cosmos, SQL DB, Fabric, SharePoint, ServiceNow…)	ADF / Fabric pipelines / custom Functions. You build and operate each connector and its change-tracking.	Build yourself

4 096-dim vector fields (supports text-embedding-3-large at 3 072)	SQL Server 2025 hard-caps at **1 998 dims**. You must use a smaller embedding model (ada-002 at 1 536 fits; 3-large at 3 072 does not).	Hard ceiling
Production-grade ANN vector index (GA)	DiskANN is **public preview** in May 2026. Exact KNN works but only practical under ~50 K vectors.	Preview risk
Native suggesters / autocomplete with fuzzy match	Trigram or prefix index, hand-built ranker.	DIY

Upfront engineering build-out (one-time, NRE)

WORKSTREAM	ENG-WEEKS (RANGE)	NOTES
Schema, Hyperscale provisioning, networking, identity	1–2	Straightforward.
Ingestion pipeline (replace indexers)	4–8	One pipeline per source system; change tracking.
Document cracking (Document Intelligence integration)	1–2	If you have rich PDF/DOCX content.
Chunking pipeline	1–2	Recursive splitter or domain-aware.
Embedding generation pipeline	1–2	Batch via Functions / Fabric. Avoid per-row `sp_invoke_external_rest_endpoint`.
DiskANN vector index setup + tuning	1–2	Preview feature; expect iteration.
Hybrid retrieval w/ RRF in T-SQL	2–4	One stored procedure per query shape; hand-coded RRF.
Semantic re-ranker (host or call AOAI)	4–8	Biggest individual line item. GPU container app + model serving + latency tuning.

Faceting, filters, scoring profiles	2–4	Per query/result shape.
Observability & relevance dashboards	2–3	You lose AI Search query telemetry; rebuild it.
Testing, NDCG/MRR benchmarking, cutover	4–8	Shadow traffic, A/B, parity sign-off.
Total	**23–45 eng-weeks**	**≈ 2 senior engineers for 12–22 calendar weeks.**

At a fully-loaded $8 K / engineer-week (US senior IC, blended), the build cost lands roughly **$184 K – $720 K one-time**. The model's default is $350 K. Add the cost-tab "SQL one-time build cost" input to fold it into the projection as straight-line amortization across your horizon.

15-step build playbook

In order. Where parity is lost, called out inline.

1. **Stand up the database**

   Provision Azure SQL Database Hyperscale (Standard Gen5 or Premium Mem-Opt). Enable Premium-series memory-optimized compute for vector workloads. Configure 1–4 HA replicas to mirror your AI Search replica count and one or more named replicas for read scale-out.

2. **Design the schema**

   Create a `documents` table for source metadata and a `chunks` table with `chunk_id`, `doc_id`, `text nvarchar(max)`, `embedding vector(N)`, plus filterable / facetable columns. Add full-text catalog on the text column.

3. **Replace indexers — build an ingestion pipeline**

   AI Search indexers (Blob, ADLS, Cosmos, SQL, Fabric, SharePoint) become Azure Data Factory pipelines, Fabric Dataflows, or Azure Functions. You own change tracking (high-water mark column or CDC).

4 Replace document cracking

Call Azure AI Document Intelligence for PDF/Office/OCR. Land raw extracted text in `documents.raw_text`. AI Search did this inline; here it's an external skill in the pipeline.

5 Replace the Text Split skill — chunking

Implement chunking in the pipeline (LangChain RecursiveCharacterTextSplitter, Semantic Kernel, or your own UDF). Persist chunks as rows in `chunks`.

6 Replace integrated vectorization — embeddings

Generate embeddings with Azure OpenAI `text-embedding-3-large`. Either call `sp_invoke_external_rest_endpoint` from a stored procedure (per-row, simple) or batch externally in the pipeline (faster, cheaper). Store as `vector(1536)` or `vector(3072)` — note SQL Server 2025 caps at 1 998 dimensions.

7 Build the vector index

`CREATE VECTOR INDEX IX_chunks_emb ON dbo.chunks(embedding) WITH (METRIC = 'cosine', TYPE = 'diskann');` Note: at GA the DiskANN index is single-replica and rebuilt from scratch — plan for batched maintenance windows.

8 Re-implement hybrid retrieval with RRF

Write a single query that runs an FTS lexical search, a vector KNN, ranks each, then combines with hand-coded Reciprocal Rank Fusion. AI Search does this in one line; here it's ~40 lines of CTE/T-SQL per query shape.

9 Build a semantic re-ranker — the biggest gap

Choose one: (a) self-host a cross-encoder (e.g. `ms-marco-MiniLM-L-12-v2`) on Azure Container Apps or AKS with GPU and call it from the app tier on the top-50 results; (b) call Azure OpenAI `gpt-4o-mini` to score relevance — much higher quality, much higher cost. Either way: no built-in equivalent to AI Search semantic ranker.

10 **Recreate scoring profiles and synonyms**

Move field-weighting, freshness boosts, and tag boosts into your T-SQL ranking expression. Use FTS thesaurus XML for synonyms; stop-lists for stop words.

11 **Recreate facets, suggesters, and autocomplete**

Facets → `GROUP BY` with indexed columns or pre-aggregated materialized views. Suggesters → prefix or trigram index on the suggester source field; query with `LIKE 'term%'`.

12 **Wire up observability & query telemetry**

AI Search Insights → use Azure SQL Database Insights, Query Performance Insight, and Application Insights from the app tier. Build a custom relevance dashboard since AI Search's per-query telemetry is not available.

13 **Security, network, and identity**

Enable TDE with customer-managed key, private endpoints, Entra-only auth, row-level security if you have multi-tenant data, and column-level encryption for PII. All native in Hyperscale.

14 **If you used agentic retrieval — rebuild it**

AI Search's agentic retrieval (query planning + reasoning tokens) has no SQL equivalent. Build it in the app tier with Semantic Kernel or LangGraph orchestrating multi-turn retrieval against your SQL hybrid search.

15 **Cut over with shadow traffic**

Run both services in parallel. Replay a representative query log against both and compare top-K results with NDCG / MRR. Only cut over when SQL hybrid + custom re-rank beats AI Search baseline on your relevance benchmark.

Biggest gaps when moving off AI Search: the semantic re-ranker, integrated vectorization, indexer-style connectors, and agentic retrieval. These are real engineering work — budget them into the cost model on the previous tab via the "SQL ops overhead"

input.

Production readiness flag (May 2026): The DiskANN *vector index* in SQL Server 2025 / Azure SQL is still **public preview** — not GA. Exact KNN (no index) is GA but only practical for < ~50K vectors. If your workload needs ANN performance on millions of vectors today, AI Search HNSW is the only GA path on Microsoft's stack. Revisit when DiskANN GAs.